

A GPU instance is two computers bolted together

Figure 1: A GPU instance is two computers bolted together

Contents

GPU Infrastructure & Hosting Open-Source Models	1
PART I — THE HARDWARE	2
Chapter 1 — A GPU instance is two computers bolted together	2
Chapter 2 — VRAM islands and the bandwidth ladder	2
Chapter 3 — Servers, the 8-GPU ceiling, and fixed shapes	2
Chapter 4 — The fixed-ratio law (why cards are bundles)	3
Chapter 5 — Memory bandwidth is the usual speed limit	3
PART II — THE MODEL	3
Chapter 6 — What “a model” actually is on disk	3
Chapter 7 — Where models live and how you fetch them	4
Chapter 8 — Sizing: will it fit, and how fast will it run?	4
PART III — THE SERVING STACK	5
Chapter 9 — The inference engine: what weights cannot do alone	5
Chapter 10 — Choosing an engine: vLLM vs Ollama vs the rest	5
Chapter 11 — The hosting lifecycle, end to end	6
Chapter 12 — Capacity: how many requests before you must scale?	6
Chapter 13 — Speed and warmth: two different latencies	7
Chapter 14 — Economics: when self-hosting wins	7
APPENDIX — GCP SPECIFICS	8
A.1 The four-level ladder, mapped to GCP	8
A.2 Startup credits: they follow the billing shape, not the model’s author	8
A.3 Model Garden decoded	8
A.4 The GLM-5.2 case study (the framework in action)	9
A.5 Break-even numbers (2K in / 300 out per request, mid-2026 prices)	9
A.6 Decision rules	9

GPU Infrastructure & Hosting Open-Source Models

Topic: Programming-Tech/infra · Type: consolidated learning note Last updated: 2026-07-16

The one-line thesis: weights are inert numbers; hardware is islands of fast memory; an inference engine is what marries the two and turns them into an API. Hosting a model = moving bytes to the right island, then keeping that island busy.

Figure 2: VRAM islands and the bridges between them

PART I — THE HARDWARE

Chapter 1 — A GPU instance is two computers bolted together

- CPU memory and GPU memory are **not two grades of one thing** — they are memory attached to **different processors**, and each processor can only compute on its own memory.
- Anything the GPU touches must first cross PCIe into VRAM. That bridge is 10–40x slower than either memory on its own side — which is why loading a model takes minutes.
- In LLM inference, “compute” means **GPU FLOPs, not vCPUs**. The host does chores (tokenize, batch, serve HTTP); it is almost never the bottleneck.
- **Server vs host:** the *server* is the whole physical box; the *host* is only its CPU side. The GPU cards are the *devices* plugged into it.

Chapter 2 — VRAM islands and the bandwidth ladder

Multiple GPU cards in one server do **not** pool their memory. Each card’s VRAM is an island; only the bridges between islands differ in speed.

The bandwidth ladder (geography analogy):

Link	Bandwidth	Analogy
On-card: VRAM its own cores	~300–3,000 GB/s	streets inside a city
NVLink: card card, same server	~900 GB/s	bridges between neighbor cities
PCIe: host card (or card card)	~64 GB/s	regional roads
Network: server server	~10–40 GB/s	highways between countries

- **PCIe** = the universal socket every computer has (GPUs, SSDs, NICs all use it). General-purpose → modest bandwidth. Office analogy: the shared corridor system.
- **NVLink** = Nvidia’s private card-to-card connector, bypassing PCIe and the CPU. One job only → ~14x faster. Analogy: a private bridge between two offices.
- Big models are **sharded** across islands (tensor parallelism: 1/8th of every layer per card), and the cards exchange partial results **every single token**. That’s why multi-GPU servers need NVLink, why nobody gangs 8 PCIe-only L4s into a fake big GPU, and why multi-*node* (network-speed) inference is the option of last resort.
- **A model’s total memory footprint tells you how far down this ladder every one of its tokens must travel.**

Chapter 3 — Servers, the 8-GPU ceiling, and fixed shapes

one CPU socket 128 PCIe lanes ÷ 16 lanes per GPU 8 cards
 one H100 700 W → 8 cards 5.6 kW per box (cooling limit)
 motherboard slots: finite
 industry-standard server = up to 8 GPUs

- Nvidia’s own DGX/HGX reference designs and the clouds’ biggest instances are 8-GPU boxes. Need more VRAM than 8 cards hold → you *must* go multi-node; there is no bigger single box to buy.

- Clouds sell **fixed shapes from a menu**, never free composition: e.g. “1/2/4/8 L4s with set vCPU ranges” or “8×H100, take it or leave it.” The host side (vCPUs, RAM) is assembled from interchangeable parts, so ratios vary a little; the card is atomic — you take all its VRAM or none.

Chapter 4 — The fixed-ratio law (why cards are bundles)

VRAM is soldered onto the GPU package. **You cannot buy compute and memory separately** — no console anywhere sells “L4 compute with 100 GB of memory.”

Card	VRAM	Compute	Note
L4	24 GB	modest	the small-model workhorse
A100	40/80GB	high	
H100	80 GB	v. high	
H200	141 GB	= H100	Nvidia shifting the ratio toward memory
RTX Pro 6000	96 GB	mid-high	

Consequences: - **Memory-hungry, compute-light models (big MoE) fit the bundle terribly** for one tenant: buying ~900 GB of VRAM forces buying 8 cards of FLOPs you can't fill alone. - Only **batching across many concurrent users** recovers the forced compute — which only providers have. The fixed compute:memory ratio is the physical law that generates the token-API (MaaS) business model. - Rule: **self-host dense small models; buy tokens for big MoE.**

Chapter 5 — Memory bandwidth is the usual speed limit

Generating each token requires re-reading essentially all active weights from VRAM.

```
per token:  cores      read ~all active weights      VRAM
              (fast)                                (the limit)
```

- Inference is typically **memory-bandwidth-bound, not FLOPs-bound**.
- Example: the L4's 300 GB/s (6.7x below an A100) is why it generates only ~50–70 tok/s single-stream on a 4B model while its tensor cores sit partly idle.
- Corollary: batching is nearly free speed — the weights are being read anyway, so amortize each read across many requests' tokens (Part III, Chapter 10).

PART II — THE MODEL

Chapter 6 — What “a model” actually is on disk

Open weights are **a folder of files**, inert without an engine:

```
hugging-face repo: Qwen/Qwen3-4B-Instruct
  config.json           ← architecture: layers, heads, dims, MoE?
  tokenizer.json        ← text   token-id mapping
  tokenizer_config.json
  generation_config.json ← default sampling settings
  model-00001-of-00002.safetensors ← the weights (just tensors)
  model-00002-of-00002.safetensors
  model.safetensors.index.json      ← which tensor lives in which shard
```

The weights’ journey from registry to VRAM

Figure 3: The weights’ journey from registry to VRAM

- **safetensors** = today's standard weight format (memory-mappable, safe to load — no arbitrary code execution, unlike old pickle .bin files).
- The weights are literally billions of numbers. They cannot tokenize, sample, batch, or serve HTTP. **Weights without an inference engine are a library without a reader.**
- Quantized variants are separate artifacts of the same model:
 - **FP16/FP8 safetensors** → vLLM/SGLang/TGI territory (GPU serving)
 - **GGUF** (single packed file) → llama.cpp/Ollama territory (CPU/GPU local)
 - **AWQ / GPTQ** safetensors → pre-quantized INT4 for GPU engines

Chapter 7 — Where models live and how you fetch them

Hugging Face Hub is the registry (think: Docker Hub for weights). Each model is a git repo with revisions; big files ride on LFS/CDN.

The tooling, bottom to top: - **huggingface_hub** (Python lib) — the downloader. Key calls: `snapshot_download("Qwen/Qwen3-4B-Instruct")` (whole repo, cached, resumable), `hf_hub_download(...)` (single file). CLI: `hf download <repo>`. - **Cache location:** `HF_HOME` / `~/.cache/huggingface/hub` — content-addressed; repeated loads hit disk, not the network. In containers, mount this as a volume or bake weights into the image/bucket, or every cold start re-downloads. - **Speed:** `pip install hf_transfer + HF_ENABLED_TRANSFER=1` for multi-gigabit downloads (a 4B model in seconds-to-a-minute; a 70B in minutes, not hours). - **Gated models** (Llama, some Gemma): accept the license on the model page once, then authenticate with an HF token (`HF_TOKEN` env var / `hf auth login`). - **transformers** (Python lib) — reads `config.json` + tokenizer + weights into a runnable PyTorch model. Fine for experiments and scripts; **not a serving engine** (no batching/scheduling). Engines like vLLM reuse its configs/tokenizers but implement their own optimized runtime. - **Production pattern:** download once → push to your object store (GCS/S3) → stream to VRAM at startup (e.g. Run:ai Model Streamer), so serving never depends on HF availability or rate limits.

Chapter 8 — Sizing: will it fit, and how fast will it run?

Weights: $\text{params} \times \text{bytes/param}$

Precision	Bytes/param	1B params	4B	26B	744B (GLM-5.2)
FP16/BF16	2	2 GB	8 GB	52 GB	~1.5 TB
FP8 / INT8	1	1 GB	4 GB	26 GB	~744 GB
INT4 (Q4)	~0.5	0.5 GB	2 GB	13 GB	~372 GB

Add 20–30% headroom for the engine’s runtime; the engine then claims the rest of the card for KV cache automatically (vLLM reserves ~90% of VRAM by default).

KV cache: the memory that grows with traffic

KV bytes per token	$2 \text{ (K and V)} \times \text{layers} \times \text{KV heads} \times \text{head_dim} \times \text{bytes}$
total cache	$\text{per-token cost} \times \text{context length} \times \text{concurrent requests}$
	the multiplier that explodes

MoE: memory scales with total params, compute with active

Figure 4: MoE: memory scales with total params, compute with active

The full hosting stack — the engine is the middle that makes weights serve

Figure 5: The full hosting stack — the engine is the middle that makes weights serve

- **GQA/MQA** (shared KV heads) cut it 4–8x; **MLA** (latent compression) more.
- 10 concurrent requests $\times$ 100K context $\times$ $\sim$ 100 KB/token = **100 GB of cache** — potentially more than the weights. This is the physics behind long-context price tiers on token APIs.

MoE: read “XB total, YB active” correctly

- Active count does **not** reduce memory — you can’t page experts you can’t predict.
- **Total** $\rightarrow$ **memory**; **active** $\rightarrow$ **speed** (and the provider’s serving cost — why a 744B MoE can be priced like a mid-size dense model per token).

Worked examples

- **Qwen3 4B @ FP8** = $\sim$ 4 GB on a 24 GB L4 $\rightarrow$ $\sim$ 17 GB free for KV cache $\rightarrow$ huge batching headroom. This is why the L4 owns the 4B class.
- **Gemma 4 26B-A4B @ FP8** 26 GB $\rightarrow$ *just* misses the L4; needs the next card up, then runs at 4B-class speed.
- **GLM-5.2 (744B MoE)** $\rightarrow$ $\sim$ 850–950 GB practical $\rightarrow$ an 8 $\times$ H200 node. Self-hosting it is a cluster problem, not a shopping problem.

The complete sizing formula: $\text{params} \times \text{precision} = \text{weights} \cdot \text{architecture} \times \text{context} \times \text{batch} = \text{cache} \cdot \text{weights} + \text{cache}$ must fit VRAM $\cdot$ MoE discounts compute, never memory.

PART III — THE SERVING STACK

Chapter 9 — The inference engine: what weights cannot do alone

The engine is the software that loads weights into VRAM and turns them into a service. Everything users experience — speed, concurrency, the API — is the engine’s work.

What the engine actually does per request: 1. **Tokenize** the prompt (on the host CPU). 2. **Schedule** it into the running batch alongside other requests. 3. **Prefill** — one big pass over the prompt, filling its KV cache. 4. **Decode** — generate token by token, re-reading weights each step. 5. **Stream** tokens back over HTTP; **evict** the KV cache when done.

Chapter 10 — Choosing an engine: vLLM vs Ollama vs the rest

Engine	Built for	Format	Killer feature	Reach for it when
vLLM	production GPU serving	safetensors	PagedAttention + continuous batching	serving many users on a GPU
Ollama	local dev, one user	GGUF	ollama run llama3 simplicity	laptop experiments, quick evals

Engine	Built for	Format	Killer feature	Reach for it when
SGLang	production, agentic	safetensors	RadixAttention (prefix cache)	heavy shared-prefix workloads
TGI	production (HF's own)	safetensors	tight HF integration	HF-centric stacks
llama.cpp	CPU/edge	GGUF	runs anywhere, no GPU needed	edge devices, CPU-only boxes
TensorRT-LLM	max performance	compiled	Nvidia-tuned kernels	squeezing the last 20% at scale

The two you named, contrasted honestly: - **Ollama** wraps llama.cpp: single-user ergonomics, GGUF quantized files, zero config. Wonderful for development; **wrong for serving** — no continuous batching, so throughput collapses the moment two users arrive. - **vLLM** is the production default: one process serves hundreds of concurrent requests from one card, exposing an OpenAI-compatible API.

vLLM's two signature ideas (worth understanding, they explain its throughput): - **PagedAttention** — manage KV cache like an OS manages RAM: in small pages, not one contiguous block per request. Kills fragmentation → ~2-4x more concurrent requests fit in the same VRAM. - **Continuous batching** — requests join and leave the running batch *every token*, instead of waiting for the whole batch to finish. GPU never idles between requests.

```
static batching:  [req1      ] [req2 waits.....]  ← wasted GPU
continuous:      [req1      ) (req3    ...
                  [req2      ) (req4    ...      ← always full
```

Chapter 11 — The hosting lifecycle, end to end

Minimal working commands, for the muscle memory:

```
# vLLM: production serving on a GPU box
pip install vllm
vllm serve Qwen/Qwen3-4B-Instruct --max-model-len 8192
# → OpenAI-compatible API on :8000 (downloads from HF automatically, caches)

curl localhost:8000/v1/chat/completions -H 'Content-Type: application/json' \
  -d '{"model": "Qwen/Qwen3-4B-Instruct", "messages": [{"role": "user", "content": "hi"}]}'

# Ollama: local development
ollama run qwen3:4b          # pulls GGUF, chats in terminal
```

Cold-start anatomy (why step 2. dominates): pull container → download/stream weights → cross PCIe → allocate cache → compile kernels. Mitigations: bake or mount the HF cache; stream weights from your own bucket; keep min-instances=1 where warmth matters.

Chapter 12 — Capacity: how many requests before you must scale?

The question every host eventually asks. The answer falls out of the sizing formula:

The hosting lifecycle, end to end

Figure 6: The hosting lifecycle, end to end

1. VRAM budget for cache = card VRAM - weights - overhead
L4 example: 24 - 4 (Qwen3-4B @FP8) - 2 = 18 GB
2. concurrent requests = cache budget ÷ (KV per token × avg context)
e.g. 18 GB ÷ (~65 KB/tok × 4K ctx = 260 MB/req) = ~70 requests in flight
3. but throughput saturates earlier: memory bandwidth is shared -
aggregate tok/s plateaus; per-request tok/s degrades as batch grows

So there are **two ceilings**, and you hit the soft one first: - **Hard ceiling (memory)**: requests no longer fit in KV cache → engine queues them. - **Soft ceiling (latency SLO)**: batch is so full that per-request speed drops below what users tolerate.

How to find YOUR number (the only honest method): load-test. 1. Fix a realistic request shape (input/output tokens from your real traffic). 2. Ramp concurrency (1, 2, 4, 8, ...) with a tool like `vllm bench serve / k6 / locust`. 3. Plot: aggregate tok/s (rises then plateaus) and p95 latency (flat then bends up). 4. **Your capacity = the concurrency just before p95 crosses your SLO.** 5. Scale trigger: alert at ~70–80% of that; add a replica behind a load balancer.

tok/s	plateau = bandwidth saturated
	p95
	SLO breached → scale
conc.	concurrency
	your capacity is here

Scaling directions, in order of preference: - **Up the batch** (free, until the soft ceiling) → **more replicas** (linear, simple) → **bigger card** (only when the model doesn't fit or single-stream speed matters) → **shard across cards** (only when nothing else fits the model at all).

Chapter 13 — Speed and warmth: two different latencies

- **Cold start** = provisioning + weight loading. Solved by warmth (min-replicas, scheduled warm-up pings). A config/money problem.
- **Generation time** = the physics of decode (memory bandwidth). Solved only by smaller models, better silicon, or fewer output tokens.
- A warm endpoint is NOT “near-zero inference”: a 4B on a warm L4 still takes ~4–6 s for a 300-token answer (~50–70 tok/s single stream).
- **Warmth is always paid for by someone.** Token APIs: the provider amortizes it across all customers (free to you). Self-hosting: you pay for idle or eat cold starts. Scale-to-zero *is* the decision to accept cold starts. No third option.
- Hosted frontier APIs on optimized fleets (~200–360 tok/s, ~0.3 s TTFT) beat any small-GPU self-host on per-request speed. **Self-hosting small GPUs is a throughput/cost play at sustained volume — never a speed play.**

Chapter 14 — Economics: when self-hosting wins

- **Usage-billed (tokens) wins at low utilization; time-billed (GPU-hours) wins at high utilization.** The crossover is a computable threshold, never an opinion: **break-even requests/month = monthly cost of always-warm GPU ÷ API cost per request**
- A GPU only beats token pricing when its **batch is actually full**. Bursty traffic on self-hosted GPUs costs more per useful token than an API.
- Before fleeing a “costly” API: exhaust its levers — batch discounts, prompt/context caching, token hygiene. Perceived expense is usually input-token bloat.

- **Quality gate before migrating:** benchmark the open model vs the API model on ~100 real prompts. Retries and bad outputs cost more than tokens.
- Spot/preemptible GPUs cut always-warm cost ~3x; pair with an API fallback route for preemption gaps (a try/except, not a project).

APPENDIX — GCP SPECIFICS

A.1 The four-level ladder, mapped to GCP

Level	What you buy	GCP surfaces	Billing
1	tokens (MaaS)	Vertex Gemini API · Gemini Batch · AI Studio · Model Garden MaaS	per MTok
2	managed endpoint	Vertex AI Inference endpoints (Model Garden deploy / Model Registry)	node-hour + mgmt fee
3	serverless container	Cloud Run + GPU (L4 24 GB, RTX Pro 6000 96 GB)	per second
4	machines	GCE GPU VMs · GKE GPU/TPU node pools	machine-hour

Batch column: Gemini Batch (L1) · Vertex Batch Prediction (L2) · Cloud Run Jobs (L3) · GCP Batch (L4). Provider-portable: Bedrock=L1, SageMaker=L2, EC2/EKS=L4.

A.2 Startup credits: they follow the billing shape, not the model’s author

- **Covered:** Gemini/Gemma at every level + ALL standard infra (Cloud Run GPU, GCE, GKE, Vertex endpoints, storage). Deployed open weights = GCP compute = covered; the author is billing-irrelevant.
- **Excluded:** third-party models billed per token (“billed directly, not covered”) and Marketplace license fees.
- Sources: cloud.google.com/startup/benefits (footnote 5), [/startup/faq](https://cloud.google.com/startup/faq), [/terms/startup-program-tos](https://cloud.google.com/startup/terms).
- **Empirical test:** one small call on the SKU → next day Billing → Cost Table → did the promotional credit offset the line?
- Corollary: the only serverless + always-warm + credit-covered combination is Gemini/Gemma via API — by design.

A.3 Model Garden decoded

A **catalog, not a service**. Three listing types — classify by billing shape, never by page layout (same “Deploy” button, different consequences): 1. **Google models** (Gemini API; Gemma as API and weights). 2. **Third-party MaaS** (Claude, GLM, Llama, DeepSeek as managed APIs) — “Deploy” just enables the API. Smallest slice, front-page placement, credit-excluded. 3. **Open-weight self-deploy** — the majority. “Deploy” provisions real GPUs in your project, billed hourly until undeployed.

Ghost-endpoint trap: L2 endpoints bill 24/7 idle; a forgotten H100 endpoint \$8K/month. Undeploy after testing; low-threshold billing alerts.

Calling third-party MaaS (e.g. GLM): POST <https://aiplatform.googleapis.com/v1/projects/{P}/locations/{L}/models/{M}:predict> with body `model zai-org/glm-5-maas` (OpenAI-compatible; different endpoint family than Gemini’s)

publishers/google/...:generateContent). Auth: OAuth2/service accounts only, ~hourly token expiry → use LiteLLM (vertex_ai/zai-org/...) or refresh logic. One-time: Model Garden → Enable → accept license form.

A.4 The GLM-5.2 case study (the framework in action)

Three independent properties of any listing: license → legal to self-host? · parameter count → feasible? · billing shape → credits? GLM-5.2: MIT legal · 744B infeasible (~900 GB → 8×H200 cluster) · MaaS per-token credits. Verdict: consume as tokens, real money, from the cheapest route — and at \$1+/\$3.2+ per MTok it competes with Claude on coding, not with Flash-Lite on cheap small tasks.

A.5 Break-even numbers (2K in / 300 out per request, mid-2026 prices)

Always-warm option	~Monthly	Break-even vs cached Flash-Lite
Cloud Run L4, min-instances=1	~\$650	~85K requests/day
Vertex endpoint, Spot L4	~\$210–240	~30K requests/day
Raw GCE Spot L4 VM (DIY ops)	~\$160–180	~20–25K requests/day

Gemini levers first: Batch −50% both directions; context caching 10% rate on cached input; GPU CUDs are weak (8–11% vs 37–55% for regular compute).

A.6 Decision rules

1. Interactive + cheap + zero ops → Gemini Flash-Lite realtime + caching (forced if “no cold start” and “credits” are both requirements).
2. Async pipeline work → Batch API, always.
3. Sustained ~25–30K small req/day + passed quality gate → 4B open model on Vertex Spot L4 endpoint or Cloud Run L4, with API fallback on preemption/429.
4. Big MoE / frontier third-party → tokens only, cheapest route (credits excluded everywhere anyway).
5. 96 GB tier → a quality decision (26B+ needed), never a cost decision.
6. Never leave an L2 endpoint deployed after testing.
7. Classify any new offering in 30 seconds: ladder level → billing unit → who pays for warmth → does the billing shape match the credits.